

Beyond the Hype: Limitations of Machine Learning in Geotechnical Engineering

Krishna Kumar

CE397 Scientific Machine Learning

The University of Texas at Austin

September 10, 2026

Contents

1	Challenges in Geotechnical Classification Models	2
1.1	Class Imbalance: The Accuracy Illusion	2
1.1.1	What Accuracy Hides	2
1.1.2	The Right Metrics for Engineering	3
1.1.3	Engineering the Solution	3
1.2	Spatial Leakage: The Dataset is Lying to You	3
1.2.1	The Mistake	3
1.2.2	The Consequences	4
1.2.3	The Correct Approach: Site-Based Splitting	4
1.2.4	Spatial Cross-Validation	5
1.2.5	Geographic Extrapolation	5
1.2.6	Poisson Disc Sampling: Enforcing Spatial Independence	5
1.3	Temporal and Instrumentation Leakage	6
1.3.1	Time Series: The Ordering Matters	6
1.3.2	Instrumentation: One Slope, Many Sensors	6
1.4	Feature Leakage: Using the Future to Predict the Past	7
1.4.1	The Check	7
1.5	Data Preprocessing Leakage	7
1.5.1	The Trap	7
1.6	Misapplication of Post-Hoc Explainability (SHAP)	8
A	Validation Framework: The Checklist	9
A.1	Data Integrity	9
A.2	Performance Metrics	9
A.3	Physical Consistency	9
A.4	Uncertainty and Limitations	9

1 Challenges in Geotechnical Classification Models

The application of machine learning to classification tasks in geotechnical engineering—such as liquefaction potential, slope failure, or soil type identification—presents unique challenges. A classifier’s goal is to map input features ($\mathbf{x} = [\text{PGA}, V_{s30}, q_c, \dots]^T$) to a discrete class ($y \in \{0, 1\}$), typically by classifying “Liquefaction” if the model’s output probability $\hat{p} > 0.5$.

This seemingly straightforward task is plagued by fundamental issues. This section addresses three critical traps: (1) class imbalance that creates an “accuracy illusion,” (2) data leakage, particularly from spatial autocorrelation, that inflates performance, and (3) the misapplication of post-hoc explainability methods as a substitute for validation.

1.1 Class Imbalance: The Accuracy Illusion

A primary challenge in geotechnical hazard modeling is the prevalence of class imbalance. High-consequence events, such as liquefaction or slope failure, are rare in datasets.

Consider a typical liquefaction database: 5,000 case histories, with 200 “liquefaction” cases (4%) and 4,800 “no-liquefaction” cases (96%). A naive classifier can achieve **96% accuracy** by simply predicting “no liquefaction” for every single site. This model, while statistically “accurate,” has learned nothing about physics and is practically useless. Yet, such results are often published.

The problem is structural. A standard cross-entropy loss function is minimized by correctly classifying the dominant class, and the rare, critical failure cases are drowned out.

1.1.1 What Accuracy Hides

Accuracy obscures what matters: the confusion matrix. For the degenerate “always predict no” model, the reality is clear in Table 1.

Table 1: Confusion Matrix for Degenerate “Always Predict No” Classifier

	Predicted: No Liq	Predicted: Liq
Actual: No Liq	4800 (True Negative)	0 (False Positive)
Actual: Liq	200 (False Negative)	0 (True Positive)

The model missed *every single liquefaction case*. In geotechnical practice, the consequences of misclassification are asymmetric:

- **False Negative (FN):** A missed hazard, potentially leading to catastrophic failure.
- **False Positive (FP):** Over-design, leading to unnecessary financial cost.

The cost of a False Negative far exceeds that of a False Positive. Standard accuracy treats them as equal.

1.1.2 The Right Metrics for Engineering

Models must be evaluated using metrics that account for this imbalance.

- **Recall:** *Of all sites that liquefied, what fraction did we catch?* $(TP/(TP + FN))$. This is the key metric for safety. The degenerate model has 0% Recall.
- **Precision:** *Of sites predicted to liquefy, what fraction actually did?* $(TP/(TP + FP))$. This is the key metric for cost.
- **F1-Score:** The harmonic mean of Precision and Recall, providing a balanced single-score.

A useful model might achieve 75% Recall with 80% Precision ($F1 = 0.77$), correctly flagging 150 of 200 liquefaction cases while managing false alarms. This is a trustworthy tool; "96% accuracy" was not.

1.1.3 Engineering the Solution

Three approaches can restore balance:

1. **Reweighting the Loss:** Penalize the errors that matter more. By setting the loss function to penalize False Negatives 25 times more than False Positives, the optimizer is forced to attend to the rare, critical events.
2. **Resampling:** Balance the training data by oversampling the minority class (e.g., SMOTE) or undersampling the majority class.
3. **Threshold Moving:** Adjust the classification threshold (e.g., from $p > 0.5$ to $p > 0.2$) to increase Recall at the expense of Precision. This threshold becomes a design choice reflecting project risk tolerance.

In geotechnical engineering, models must be optimized for **Recall at an acceptable Precision**, not for accuracy.

1.2 Spatial Leakage: The Dataset is Lying to You

This is the most insidious error in geotechnical ML. It inflates performance by 10-20% and is present in most published papers. Here is how it happens.

1.2.1 The Mistake

You collect 1000 CPT soundings from 50 project sites. Standard practice:

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.2)
```

This randomly assigns 800 soundings to training, 200 to test. You train a classifier for soil type or liquefaction. Test accuracy: 92%. Paper submitted.

The test set is not independent. Site A contributed 30 soundings. Random split puts ~ 24 in training, ~ 6 in test. But soil properties at Site A are spatially correlated over 50-100m. A test sounding at (x=35m, y=12m) sits 20m from a training sounding at (x=15m, y=8m). Same geologic unit, same soil. The model memorized Site A's geology—it did not learn to generalize to new sites.

Spatial autocorrelation is fundamental to geotechnical engineering. The variogram of V_s or q_c shows correlation lengths of 10-100m horizontally. Data points within this range are not independent observations. Treating them as such during train/test splitting violates the assumption underlying all validation: test data must be independent of training data.

1.2.2 The Consequences

A model validated with spatial leakage fails spectacularly on truly new sites. Example from practice:

- **Training:** 5 sites in San Francisco Bay Area (soft bay mud)
- **Random split validation:** 95% accuracy (leaked!)
- **Deployment:** Applied to Seattle site (glacial till)
- **Actual performance:** 58% accuracy (barely better than a coin flip)

The model learned “if you are in the Bay Area and $q_c < 2$ MPa, it is bay mud.” It never learned general soil mechanics. Different geology $\rightarrow$ total failure.

1.2.3 The Correct Approach: Site-Based Splitting

Split at the **project level**, not the sounding level:

```
# Group by site
sites = data.groupby('site_id')

# Randomly assign entire sites to train/test
train_sites = sites.sample(frac=0.8)
test_sites = sites.drop(train_sites.index)

X_train = data[data['site_id'].isin(train_sites.groups.keys())]
X_test = data[data['site_id'].isin(test_sites.groups.keys())]
```

Now test sites are completely unseen during training. Performance drops from 92% to 76%—that is the **honest estimate** of how the model will perform on a new project.

Table 2: Leave-One-Site-Out Cross-Validation Results

Fold	Training Sites	Test Site	Accuracy
1	B, C, D, E	A	78%
2	A, C, D, E	B	74%
3	A, B, D, E	C	71%
4	A, B, C, E	D	80%
5	A, B, C, D	E	69%
Average			74%

1.2.4 Spatial Cross-Validation

For small numbers of sites, use leave-one-site-out cross-validation:

Average: 74%. Report this, not the 92% from random splitting. If Site E gives 69%, investigate: different geology? Model limitation? At least you know.

1.2.5 Geographic Extrapolation

The ultimate test: train on one region, test on another. Example:

- **Training:** All California sites (n=40)
- **Validation:** Washington sites (n=10)

If performance degrades severely, the model learned regional geology, not universal soil behavior. This is fine—just document it. Market as “California liquefaction model (accuracy: 82% in CA, 65% in PNW).” Honesty matters more than high numbers.

1.2.6 Poisson Disc Sampling: Enforcing Spatial Independence

An alternative to site-based splitting is Poisson disc sampling, which ensures minimum distance between training and test points. This is particularly useful when sites are not well-defined or when soundings are scattered irregularly across a region.

The principle: No two points (one from training, one from test) can be closer than a threshold distance $r_{\min}$, where $r_{\min}$ is chosen based on the spatial correlation length of soil properties.

Choosing $r_{\min}$:

The threshold should exceed the spatial correlation range. For typical geotechnical properties:

- **Horizontal correlation length:** 50-100m for sedimentary deposits
- **Vertical correlation length:** 1-5m (much shorter)
- **Safe choice:** $r_{\min} = 2 \times \lambda_{\text{horizontal}} \approx 200\text{m}$

Table 3: Comparison of Spatial Validation Methods

Method	Advantage	Disadvantage
Site-based	Conceptually simple, reflects project boundaries	Requires well-defined sites, coarse granularity
Poisson disc	Works with scattered data, precise control of separation	More complex algorithm, may not use all data
Geographic regions	Tests true spatial extrapolation	Requires enough data in multiple regions

This ensures test points are in statistically independent regions from training points.

Comparison with site-based splitting:

The key insight: spatial independence is not binary (same site vs different site) but continuous (distance-dependent correlation). Poisson disc sampling respects this continuum by enforcing minimum separation rather than categorical boundaries.

1.3 Temporal and Instrumentation Leakage

1.3.1 Time Series: The Ordering Matters

Consider consolidation settlement monitoring. Measurements at $t = [0, 30, 60, 90, 180, 365]$ days. Goal: predict if settlement exceeds threshold at $t = 365$.

Wrong:

```
train_test_split(all_time_points, test_size=0.2)
```

Some $t = 180$ points in training, some $t = 365$ points in test, all from the same projects. The model learns to interpolate between nearby time points. Easy problem, high accuracy, useless predictor.

Correct:

```
# Train on t <= 180 days
X_train = data[data['time'] <= 180]
X_test = data[data['time'] == 365]
```

Now the model must **extrapolate to the future**—the real task. Accuracy drops because this is harder. That is reality.

1.3.2 Instrumentation: One Slope, Many Sensors

A slope has 20 inclinometers. They all measure the **same failure event**. If you put 16 in training and 4 in test, they are not independent—they share the common cause (slope stability, rainfall history, soil properties). One slope = one data point.

Correct approach: Use slope-level splitting. Each slope (with all its sensors) is entirely in training or entirely in test. This is analogous to site-based splitting for spatial data.

1.4 Feature Leakage: Using the Future to Predict the Past

Feature leakage occurs when an input variable contains information about the outcome that would not be available at prediction time.

Egregious example: Predicting liquefaction using post-earthquake ground settlement as an input feature. Settlement **is** liquefaction’s manifestation. You are predicting the outcome from itself. Result: 99% accuracy, zero value.

Subtle example: Predicting slope failure using displacement rate from inclinometers. If displacement is accelerating, failure is imminent or already occurring. The feature contains the outcome. This gives high accuracy for **detection** (yes, the slope is failing), but zero predictive power (we cannot forecast failure before it starts).

1.4.1 The Check

For every input feature, ask:

1. Is this known **before** the prediction time?
2. Is this **independent** of the outcome?
3. Would this be available for a **new, unseen case**?

If “no” to any question, you have feature leakage. Remove the feature and accept lower accuracy—it was illusory anyway.

1.5 Data Preprocessing Leakage

This is subtle but pervasive. Data preprocessing (normalization, feature selection, outlier removal) must happen **after** train/test split, using **training data statistics only**.

1.5.1 The Trap

Wrong order:

```
# Normalize all data
X = (X - X.mean()) / X.std()

# Then split
X_train, X_test = train_test_split(X)
```

The mean and standard deviation were computed using test data. Information leaked from test into training normalization. For small datasets, this inflates test performance by 5-10%.

Correct order:

```
# Split first
X_train, X_test = train_test_split(X)

# Compute statistics from training only
```

```
mean_train = X_train.mean()
std_train = X_train.std()

# Apply to both sets
X_train = (X_train - mean_train) / std_train
X_test = (X_test - mean_train) / std_train # Use training
statistics!
```

The test set is normalized using training statistics, as it would be in deployment. No leakage. This principle extends to: feature selection, outlier removal, oversampling (e.g., SMOTE), and dimensionality reduction (e.g., PCA).

1.6 Misapplication of Post-Hoc Explainability (SHAP)

To combat the "black box" nature of models, post-hoc explainability methods like SHAP (SHapley Additive exPlanations) are increasingly used to show which features "drive" a prediction. The common misapplication is to use SHAP as a substitute for rigorous validation. This leads to a logical fallacy: a model trained with flawed methodology (e.g., spatial leakage) achieves inflated performance, and when SHAP shows "plausible" feature importance (e.g., PGA is important), researchers incorrectly conclude the model is valid. This fails to recognize that **SHAP explains *what the model learned*, not *what is physically true***. If the model learned to exploit spatial leakage or to extrapolate non-physically (e.g., linear settlement), SHAP will faithfully explain that flawed logic, not flag it as wrong.

SHAP's correct application is as a *diagnostic* tool, not a validator, and it must only be used *after* rigorous validation is complete. The correct workflow is: (1) Build the model with proper methodology (site-based splitting, class balancing); (2) Honestly establish its generalization performance (e.g., 76% F1-score on new sites); (3) **Then**, use SHAP to confirm that this *generalizable* model is learning physically-plausible relationships. Explainability cannot compensate for, or prove, a model's validity; only rigorous validation on independent data can. In safety-critical applications, interpretable-by-design models (e.g., Logistic Regression, simple Decision Trees) are preferable to those requiring post-hoc explanation.

A Validation Framework: The Checklist

Before publishing or deploying a geotechnical classification model, verify:

A.1 Data Integrity

- ☐ Train/test split at **site or project level** (not random)
- ☐ Features available **before** prediction time (no leakage)
- ☐ Preprocessing applied **after** split using training statistics only
- ☐ Class imbalance addressed (weighting, resampling, or threshold)

A.2 Performance Metrics

- ☐ Confusion matrix reported (show all four outcomes)
- ☐ Precision, recall, and F1-score reported (not just accuracy)
- ☐ Performance on **new sites** reported separately from within-site
- ☐ False negative rate emphasized for safety-critical applications

A.3 Physical Consistency

- ☐ Predictions checked for impossible values
- ☐ Behavior tested at physical limits (e.g., $\text{PGA} \rightarrow 0$ should give $P(\text{liq}) \rightarrow 0$)
- ☐ Feature importance aligns with domain knowledge (SHAP check)
- ☐ Comparison with physics-based method or empirical correlation

A.4 Uncertainty and Limitations

- ☐ Confidence intervals or probability distributions reported
- ☐ Extrapolation regime identified (where training data sparse)
- ☐ Geographic or geologic limits stated (model trained on X, may not work on Y)
- ☐ Failed cases analyzed (not just successes)

If you cannot check all boxes, state clearly what is missing and why. A model with documented limitations is more trustworthy than one claiming unrealistic perfection.